

Masked Language Modeling

Nhóm 4

Ngày 20 tháng 4 năm 2026

Trần Lê Anh Pha — MSSV: 24521287

Trịnh Duy Hưng — MSSV: 24520610

- 1 Introduction
- 2 The architecture for bidirectional masked models
- 3 Introduction to Training Bidirectional Encoders
- 4 Masking Words
- 5 Next Sentences Prediction
- 6 Training Regimes

Giới thiệu BERT và Masked Language Modeling

Bidirectional Transformer Encoders (BERT): là mô hình ngôn ngữ dựa trên Transformer, đọc văn bản theo hai chiều (trái + phải) và được huấn luyện bằng **Masked Language Modeling**.

Masked Language Modeling (MLM): là **phương pháp huấn luyện mô hình ngôn ngữ** bằng cách **che (mask)** một số từ trong câu rồi bắt mô hình đoán lại những từ đó

Ưu điểm của BERT:

- Hiểu ngữ cảnh hai chiều (trái + phải), nắm bắt ý nghĩa chính xác hơn
- Biểu diễn từ theo contextualized embeddings (là vector biểu diễn của một từ phụ thuộc vào ngữ cảnh xung quanh nó)
- Hiệu suất cao trên các bài toán NLP: Phân loại văn bản (Text Classification), Nhận dạng thực thể (Named Entity Recognition - NER), Trả lời câu hỏi (Question Answering), Suy luận ngôn ngữ (Natural Language Inference - NLI), ...

Mục lục

- 1 Introduction
- 2 The architecture for bidirectional masked models**
- 3 Introduction to Training Bidirectional Encoders
- 4 Masking Words
- 5 Next Sentences Prediction
- 6 Training Regimes

Causal vs Bidirectional Attention

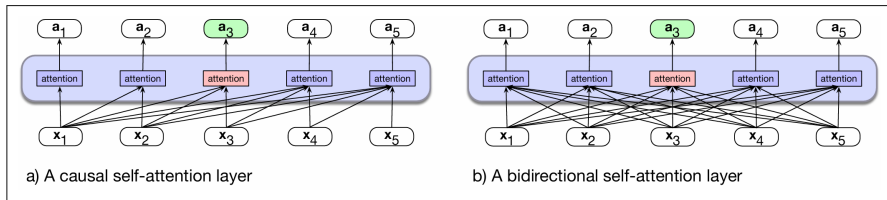
Causal:

Chỉ nhìn các token trước đó, không thấy thông tin phía sau
Dùng mask để chặn nhìn tương lai

Bidirectional:

Nhìn toàn bộ câu, hiểu ngữ cảnh đầy đủ hơn
Bỏ mask → attention trên toàn bộ input

Illustration



(a) Causal minh họa phép tính attention tại token thứ 3. Giá trị attention tại mỗi token chỉ được tính dựa trên thông tin đã xuất hiện trước đó trong ngữ cảnh.

(b) Luồng thông tin trong mô hình attention hai chiều. Khi xử lý mỗi token, mô hình có thể chú ý đến toàn bộ đầu vào, cả phía trước lẫn phía sau token hiện tại. Vì vậy, attention của token thứ 3 có thể sử dụng thông tin từ các token phía sau.

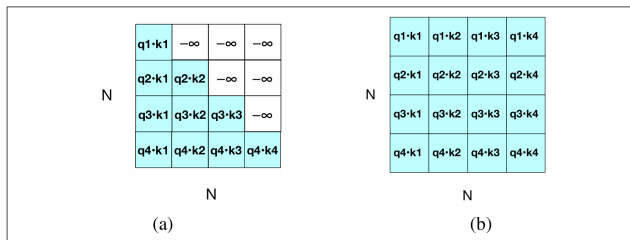
Implement

Loại bỏ bước attention masking mà đã được giới thiệu trong phương trình trước đó.

Ở chương 8, ta phải áp dụng mask lên ma trận QK^T trong các Transformer casual để đảm bảo rằng attention không thể nhìn vào các token trong tương lai:

$$\text{head} = \text{softmax} \left(\text{mask} \left(\frac{QK}{\sqrt{d_k}} \right) \right) V$$

Illustration



Hình: Enter Caption

Ma trận QK^T kích thước $N \times N$ biểu diễn các giá trị $q_i \cdot k_j$.

(a) minh họa phần tam giác phía trên của ma trận so sánh bị đưa về 0 (thực chất là gán giá trị $-\infty$, để sau khi qua hàm softmax sẽ trở thành 0), trong khi (b) minh họa phiên bản không bị che (unmasked) của ma trận.

Phép tính **attention** và kiến trúc **Transformer** (feedforward, layer norm, ...) tương tự như chương 8.

Đầu vào của mô hình là chuỗi các **subword token**, được tạo bởi các thuật toán tokenization như BPE, WordPiece hoặc SentencePiece.

Mọi câu đầu vào phải được token hóa trước, và toàn bộ quá trình xử lý diễn ra trên subword thay vì từ hoàn chỉnh.

Ví dụ: kiến trúc BERT

Vocabulary: 30,000 tokens, được tạo bằng WordPiece

Input: 512 tokens

Hidden size: 768

12 layers Transformer

12 attention heads

100M parameters

Ví dụ: kiến trúc XLNet-RoBERTa

Vocabulary: 250,000 tokens được train bằng SentencePiece Unigram LM algorithm

Input: 512 tokens

Hidden size: 1024

24 layers Transformer

16 attention heads

550M parameters

Mục lục

- 1 Introduction
- 2 The architecture for bidirectional masked models
- 3 Introduction to Training Bidirectional Encoders**
- 4 Masking Words
- 5 Next Sentences Prediction
- 6 Training Regimes

Trong quá trình huấn luyện, mô hình sẽ **che đi** một hoặc nhiều token của văn bản đầu vào, sau đó tạo ra một **phân phối xác suất trên từ vựng** cho mỗi chỗ bị thiếu (**cloze task**)

Cách tiếp cận này có thể được tổng quát hóa thành nhiều phương pháp khác nhau, trong đó bao gồm: che (mask), thay thế (substitution), đảo thứ tự (reordering), xóa (deletion), chèn thêm các phần không liên quan, ...

Tên gọi chung cho kiểu huấn luyện này là **denoising**: cố ý làm nhiễu đầu vào theo một cách nào đó rồi loại bỏ phần nhiễu đó.

Câu hoàn chỉnh: *“The water of Walden Pond is so beautiful.”*

Mô hình sẽ dự đoán từ ở vị trí:

The _____ of Walden Pond is so beautiful.

Mục lục

- 1 Introduction
- 2 The architecture for bidirectional masked models
- 3 Introduction to Training Bidirectional Encoders
- 4 Masking Words**
- 5 Next Sentences Prediction
- 6 Training Regimes

Masking Words

Chọn ngẫu nhiên token để biến đổi bởi cơ chế masking

Trong training MLM, một tỷ lệ phần trăm các token ược chọn ngẫu nhiên để được xử lý bởi **mask** (vd: 15% token ở BERT model)

Cho một câu đầu vào **lunch was delicious** và giả sử ta chọn ngẫu nhiên token thứ 3 **delicious** để xử lý:

80% số lần: Token được thay thế bằng token đặc biệt trong từ vựng có tên [MASK], ví dụ:

lunch was delicious → lunch was [MASK].

10% số lần: Token được thay thế bằng một token khác, được lấy ngẫu nhiên từ từ vựng dựa trên xác suất unigram, ví dụ:

lunch was delicious → lunch was gasp.

10% số lần: Token được giữ nguyên không thay đổi, ví dụ:

lunch was delicious → lunch was delicious.

Giải thích vì sao có 3 thao tác

Nếu ta chỉ thay thế các token bằng [MASK], mô hình có thể chỉ học cách dự đoán token khi nhìn thấy [MASK]. Nhưng mục tiêu của chúng ta là mô hình có thể cố gắng dự đoán token đầu vào trong mọi trường hợp, không phụ thuộc vào việc có xuất hiện [MASK] hay không.

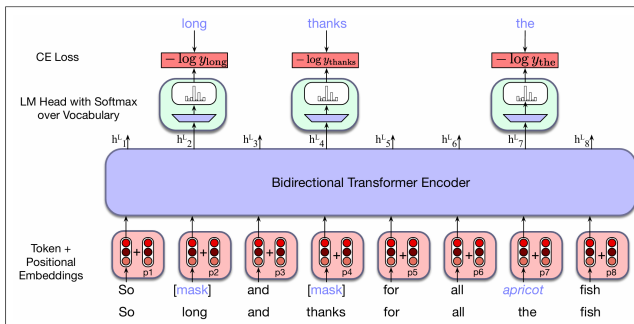
Để huấn luyện mô hình thực hiện dự đoán, chuỗi đầu vào ban đầu được token hóa bằng mô hình subword, sau đó một số token được chọn để thao tác.

Embedding của tất cả các token được lấy từ ma trận embedding E và kết hợp với positional embedding để tạo thành đầu vào cho Transformer. Đầu vào này được đưa qua các lớp bidirectional Transformer, rồi đến language modeling head.

Mục tiêu huấn luyện của MLM (Masked Language Modeling) là dự đoán lại các token gốc cho những token đã bị che. Hàm mất mát cross-entropy từ các dự đoán này được dùng để cập nhật toàn bộ tham số của mô hình.

Nói cách khác, tất cả các token đều tham gia vào quá trình self-attention, nhưng chỉ các token được chọn (bị thao tác) mới được dùng để tính loss và học.

Hình minh họa -> giải thích



Hình: Huấn luyện mô hình ngôn ngữ MLM. Trong ví dụ này, ba token trong đầu vào được chọn, trong đó hai token bị che (mask) và token thứ ba được thay thế bằng một từ không liên quan. Xác suất mà mô hình gán cho ba token này sẽ được dùng để tính loss huấn luyện. Năm token còn lại không tham gia vào việc tính loss.

Giá trị logit u được tính bằng công thức:

$$u_i = h_i^L E^T$$

$$y_i = \text{softmax}(u_i)$$

Từ đó, ta dùng cross-entropy loss để tính hàm mất mát:

$$L_{MLM}(x_i) = -\log P(x_i | h_i^L)$$

$$L_{MLM} = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i | h_i^L)$$

Mục lục

- 1 Introduction
- 2 The architecture for bidirectional masked models
- 3 Introduction to Training Bidirectional Encoders
- 4 Masking Words
- 5 Next Sentences Prediction**
- 6 Training Regimes

Next Sentence Prediction

Next Sentence Prediction (NSP): dự đoán xem các cặp câu có phải là hai câu liên tiếp thực sự trong tập dữ liệu huấn luyện hay chỉ là hai câu không liên quan.

Trong BERT, 50% các cặp huấn luyện là cặp đúng (hai câu thực sự đứng cạnh nhau), và 50% còn lại thì câu thứ hai được chọn ngẫu nhiên từ nơi khác trong tập dữ liệu.

Để hỗ trợ việc huấn luyện NSP, BERT đưa vào hai token đặc biệt trong biểu diễn đầu vào (những token này cũng rất hữu ích trong giai đoạn fine-tuning).

Token [CLS] được thêm vào đầu cặp câu đầu vào, và token [SEP] được đặt giữa hai câu và sau token cuối cùng của câu thứ hai.

Cụ thể, mỗi token trong đầu vào X thực chất được tạo bằng cách cộng ba loại embedding:

- embedding của từ (word embedding)

- embedding vị trí (position embedding)

- embedding phân đoạn (first/second segment embedding)

Trong quá trình huấn luyện, vector đầu ra h_{CLS}^L từ lớp cuối cùng, tương ứng với token [CLS], được sử dụng để biểu diễn cho nhiệm vụ dự đoán câu tiếp theo (NSP).

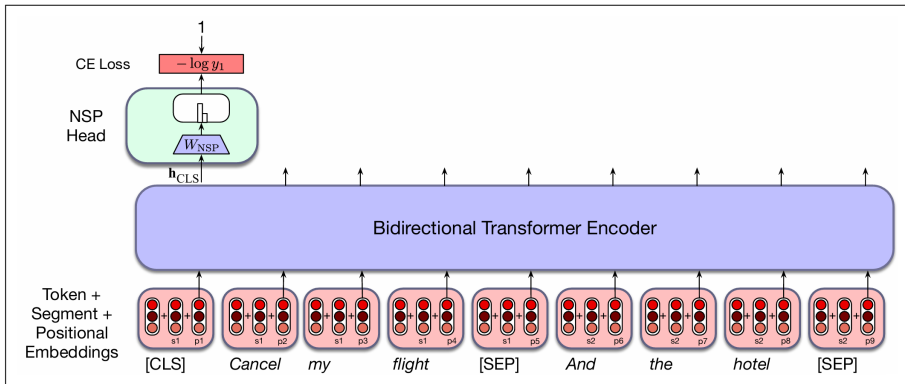
Ta thêm một lớp phân loại đặc biệt (NSP head) với trọng số $W_{\text{NSP}} \in \mathbb{R}^{d \times 2}$ để tạo ra dự đoán hai lớp:

$$y_i = \text{softmax}(h_{\text{CLS}}^L W_{\text{NSP}})$$

Hàm mất mát cuối cùng là tổng của NSP loss và MLM loss:

$$\mathcal{L} = \mathcal{L}_{\text{NSP}} + \mathcal{L}_{\text{MLM}}$$

Minh họa



Hình: An example of the NSP loss calculation.

Mục lục

- 1 Introduction
- 2 The architecture for bidirectional masked models
- 3 Introduction to Training Bidirectional Encoders
- 4 Masking Words
- 5 Next Sentences Prediction
- 6 Training Regimes**

Training regimes

Các mô hình masked language model hiện đại ngày nay được huấn luyện trên các tập dữ liệu lớn hơn nhiều từ văn bản web, đã được lọc bớt và bổ sung bằng các nguồn chất lượng cao.

Các mô hình đa ngôn ngữ cũng sử dụng văn bản web và Wikipedia đa ngôn ngữ. Ví dụ, mô hình XLM-R được huấn luyện trên khoảng 300 tỷ token thuộc 100 ngôn ngữ, lấy từ web thông qua Common Crawl.

Training regimes

Để huấn luyện BERT ban đầu, các cặp đoạn văn bản được chọn từ tập dữ liệu theo cơ chế NSP với tỷ lệ 50/50. Các cặp này được lấy sao cho tổng độ dài không vượt quá 512 token (giới hạn đầu vào của mô hình).

Sau đó, các token trong những cặp câu này được mask theo phương pháp MLM, và loss tổng được tính bằng cách kết hợp giữa mục tiêu MLM và NSP.

Vì loss này được lan truyền ngược (backpropagation) qua toàn bộ mạng Transformer, nên các embedding ở mỗi lớp sẽ học được những biểu diễn hữu ích để dự đoán từ dựa trên ngữ cảnh xung quanh.

Do token [CLS] là đầu vào trực tiếp của bộ phân loại NSP, nên biểu diễn học được của nó có xu hướng chứa thông tin về toàn bộ chuỗi câu.

Thay vì lấy các cặp câu, đầu vào sẽ chỉ là một chuỗi các câu liên tiếp, vẫn bắt đầu bằng token đặc biệt [CLS]. Nếu văn bản kết thúc trước khi đạt 512 token, một token phân tách bổ sung sẽ được thêm vào, và các câu từ tài liệu tiếp theo sẽ được “nhồi” vào cho đến khi đạt tổng cộng 512 token. Thông thường, người ta sử dụng batch size rất lớn, từ 8K đến 32K token.

Như đã biết, tất cả các mô hình ngôn ngữ đều sử dụng subword tokenization (hai thuật toán phổ biến nhất là BPE hoặc SentencePiece Unigram LM).

Vấn đề đặt ra là: nên dùng văn bản nào để học tokenization đa ngôn ngữ, khi mà một số ngôn ngữ có rất nhiều dữ liệu, còn một số khác thì rất ít?

Cách đơn giản: tạo tập dữ liệu học từ vựng bằng cách lấy ngẫu nhiên các câu từ tập huấn luyện (ví dụ từ Common Crawl), nhưng các token học được sẽ bị thiên lệch về tiếng Anh thay vì tạo ra token phổ biến cho các ngôn ngữ ít dữ liệu.

Cách tốt hơn: Chia dữ liệu thành N tập con (mỗi ngôn ngữ một tập), sau đó điều chỉnh lại xác suất chọn câu để tăng xác suất cho các ngôn ngữ ít dữ liệu.

Công thức điều chỉnh xác suất:

$$q_i = \frac{p_i^\alpha}{\sum_{j=1}^N p_j^\alpha}, \quad \text{với } p_i = \frac{n_i}{\sum_{k=1}^N n_k}$$

Nếu $0 < \alpha < 1$ sẽ tăng trọng số cho các ngôn ngữ có xác suất nhỏ (ít dữ liệu)

$\alpha = 0.3$ hoạt động rất hiệu quả, cải thiện hiệu suất đa ngôn ngữ tổng thể

Đầu ra của quá trình tiền huấn luyện này bao gồm cả các embedding từ (word embeddings) đã được học, cũng như toàn bộ các tham số của bộ mã hóa hai chiều (bidirectional encoder), được sử dụng để tạo ra các embedding theo ngữ cảnh (contextual embeddings) cho các đầu vào mới.

Ưu điểm của mô hình đa ngôn ngữ:

Không cần xây dựng nhiều mô hình đơn ngôn ngữ

Cải thiện hiệu suất cho ngôn ngữ ít dữ liệu (low-resource)

Hạn chế:

Curse of multilinguality: nhiều ngôn ngữ → giảm hiệu suất mỗi ngôn ngữ

“Accent effect”: ngôn ngữ mạnh (như tiếng Anh) ảnh hưởng ngôn ngữ yếu